



Python Programming Fundamentals

Modules and Imports

Programming Fundamentals Team

SETU

2026-09-22



Outline

1. What Is a Module?
2. Built-in vs Library Functions
3. Importing modules
4. Your Own File Is a Module
5. The Same-Folder Rule
6. Worked Example — Splitting a Program
7. Recap



Outline

- 1. What Is a Module?**
2. Built-in vs Library Functions
3. Importing modules
4. Your Own File Is a Module
5. The Same-Folder Rule
6. Worked Example — Splitting a Program
7. Recap



1. What Is a Module?

A **module** is simply a Python file — a `.py` file containing functions, constants, or classes that other files can use.



1. What Is a Module?

A **module** is simply a Python file — a `.py` file containing functions, constants, or classes that other files can use.

You have already met the idea twice:

- In the last talk, **constants** such as `PI` and `GRAVITY` were described as living at **module level** — the top of the file
- Some functions (`print`, `len`, `input`) are always available; others are not



1. What Is a Module?

A **module** is simply a Python file — a `.py` file containing functions, constants, or classes that other files can use.

You have already met the idea twice:

- In the last talk, **constants** such as `PI` and `GRAVITY` were described as living at **module level** — the top of the file
- Some functions (`print`, `len`, `input`) are always available; others are not

The big idea: every `.py` file you write is a module. Once you can import one file into another, you can split a large program into small, readable files — and reuse the functions you already wrote instead of copying and pasting them.



Outline

1. What Is a Module?
- 2. Built-in vs Library Functions**
3. Importing modules
4. Your Own File Is a Module
5. The Same-Folder Rule
6. Worked Example — Splitting a Program
7. Recap



2. Built-in vs Library Functions

Python gives you a small set of **built-in** functions with no effort at all:

```
print("hello")  
length = len("Waterford")  
value = int("42")
```



2. Built-in vs Library Functions

But most of Python's power lives in the **standard library** — hundreds of modules that ship with Python but are **not** loaded until you ask for them:

Module	What it gives you
math	pi, sqrt, floor, ceil, trigonometry
random	randint, choice, shuffle
turtle	Simple drawing and graphics
csv	Reading and writing CSV files
json	Saving and loading structured data



2. Built-in vs Library Functions

Without an import, Python does not know the name:

```
print(sqrt(16))
```

```
NameError: name 'sqrt' is not defined
```



2. Built-in vs Library Functions

Without an import, Python does not know the name:

```
print(sqrt(16))
```

```
NameError: name 'sqrt' is not defined
```

The fix is to **import the module that defines it**.



Outline

1. What Is a Module?
2. Built-in vs Library Functions
- 3. Importing modules**
4. Your Own File Is a Module
5. The Same-Folder Rule
6. Worked Example — Splitting a Program
7. Recap



3. Importing modules

We can either import the whole module, or just a few names from it.



3.1 Importing full modules:

```
import math

radius = 5
area = math.pi * radius ** 2

print(f"Area: {area:.2f}")      # Area: 78.54
print(math.sqrt(16))           # 4.0
print(math.floor(9.7))         # 9
```



3.1 Importing full modules:

```
import math

radius = 5
area = math.pi * radius ** 2

print(f"Area: {area:.2f}")      # Area: 78.54
print(math.sqrt(16))           # 4.0
print(math.floor(9.7))         # 9
```

- The import line goes at the **very top** of the file
- You then use `module_name.function_name()` — the dot says “look inside math”
- `math.pi` is a **constant** the module defines for you



3.1 Importing full modules:

The same pattern with random:

```
import random

secret = random.randint(1, 100)
greeting = random.choice(["Hi", "Hello", "Hey"])

print(greeting)
```

NOTE: `random.randint(1, 100)` includes **both** endpoints — unlike `range(1, 100)`, which stops at 99. Modules will not always behave the way the rest of the language does, so read the documentation.



3.2 Importing individual names

`from module import name`

You can pull individual names **out** of a module, so you use them without the prefix:

```
from math import pi, sqrt

radius = 5
area = pi * radius ** 2          # no "math." needed

print(f"Area: {area:.2f}")
print(sqrt(16))
```



3.2 Importing individual names

```
from module import name
```

You can pull individual names **out** of a module, so you use them without the prefix:

```
from math import pi, sqrt

radius = 5
area    = pi * radius ** 2      # no "math." needed

print(f"Area: {area:.2f}")
print(sqrt(16))
```

Same result, different style. So which should you use?



3.2 Importing individual names

	<code>import math</code>	<code>from math import pi</code>
Written as	<code>math.pi</code>	<code>pi</code>
Reads as	Clear where it came from	Shorter, but origin hidden
Risk	None	Can clash with your own names



3.2 Importing individual names

The clash is the real danger:

```
from math import pi

pi = 3                # oops – your own variable wins from here on
print(pi * 4)         # 12, not 12.57
```



3.2 Importing individual names

House rule for this module: prefer `import math` and write `math.pi`. It is a few more characters, and anyone reading your code can see instantly where the name came from. Use `from ... import` when you need just one or two names and the source is obvious.



3.3 Renaming on Import (as)

Occasionally a module name is long, or clashes with something. You can rename it as you import:

```
import random as rnd  
  
print(rnd.randint(1, 6))
```



3.3 Renaming on Import (as)

Occasionally a module name is long, or clashes with something. You can rename it as you import:

```
import random as rnd  
  
print(rnd.randint(1, 6))
```

You will see this constantly in data work (`import pandas as pd`). For now, know what it means when you see it — you rarely need it yourself.



Outline

1. What Is a Module?
2. Built-in vs Library Functions
3. Importing modules
- 4. Your Own File Is a Module**
5. The Same-Folder Rule
6. Worked Example — Splitting a Program
7. Recap



4. Your Own File Is a Module

Suppose you write `shapes.py`:

```
"""Functions for calculating areas and perimeters."""
import math

def circle_area(radius):
    """Return the area of a circle with the given radius."""
    return math.pi * radius ** 2

def rectangle_area(width, height):
    """Return the area of a rectangle."""
    return width * height
```



4. Your Own File Is a Module

Suppose you write `shapes.py`:

```
"""Functions for calculating areas and perimeters."""
import math

def circle_area(radius):
    """Return the area of a circle with the given radius."""
    return math.pi * radius ** 2

def rectangle_area(width, height):
    """Return the area of a rectangle."""
    return width * height
```

Notice: `shapes.py` **itself** imports `math`. Modules import other modules — that is normal.



4. Your Own File Is a Module

Now a **second** file, `test_shapes.py`, can use those functions:

```
import shapes

print(shapes.circle_area(3))          # 28.274333882308138
print(shapes.rectangle_area(4, 5))    # 20

print(f"Circle r=3:   {shapes.circle_area(3):.2f}")
print(f"Rect 4x5:     {shapes.rectangle_area(4, 5):.2f}")
```



4. Your Own File Is a Module

Now a **second** file, `test_shapes.py`, can use those functions:

```
import shapes

print(shapes.circle_area(3))          # 28.274333882308138
print(shapes.rectangle_area(4, 5))    # 20

print(f"Circle r=3:   {shapes.circle_area(3):.2f}")
print(f"Rect 4x5:     {shapes.rectangle_area(4, 5):.2f}")
```

- `import shapes` — the **file name** without the `.py`
- `shapes.circle_area(3)` — the same dot notation as `math.sqrt(16)`



4. Your Own File Is a Module

Now a **second** file, `test_shapes.py`, can use those functions:

```
import shapes

print(shapes.circle_area(3))          # 28.274333882308138
print(shapes.rectangle_area(4, 5))    # 20

print(f"Circle r=3:   {shapes.circle_area(3):.2f}")
print(f"Rect 4x5:     {shapes.rectangle_area(4, 5):.2f}")
```

- `import shapes` — the **file name** without the `.py`
- `shapes.circle_area(3)` — the same dot notation as `math.sqrt(16)`

There is nothing special about `math` or `random`. They are files someone else wrote. Yours work exactly the same way.



Outline

1. What Is a Module?
2. Built-in vs Library Functions
3. Importing modules
4. Your Own File Is a Module
- 5. The Same-Folder Rule**
6. Worked Example — Splitting a Program
7. Recap



5. The Same-Folder Rule

For `import shapes` to work, both files must sit in the **same folder**:

```
lab03/  
├── shapes.py      ← defines the functions  
└── test_shapes.py ← imports and uses them
```



5. The Same-Folder Rule

For `import shapes` to work, both files must sit in the **same folder**:

```
lab03/  
├── shapes.py          ← defines the functions  
└── test_shapes.py     ← imports and uses them
```

Then, in VS Code, run `test_shapes.py`. Python looks for `shapes.py` **beside the file being run**.



5. The Same-Folder Rule

```
ModuleNotFoundError: No module named 'shapes'
```

Most of the time, this means one of:

- The two files are in **different** folders
- The file name is misspelled (`Shapes.py` is not `shapes.py`)
- You are running the file from the wrong working directory



5.1 A Common Error and how to fix it

Never name your own file after a module you want to use:

```
lab03/  
├─ random.py      ← YOUR file  
└─ game.py        ← contains: import random
```



5.1 A Common Error and how to fix it

Never name your own file after a module you want to use:

```
lab03/  
├─ random.py      ← YOUR file  
└─ game.py        ← contains: import random
```

```
# game.py  
import random  
print(random.randint(1, 10))
```



5.1 A Common Error and how to fix it

```
AttributeError: module 'random' has no attribute 'randint'
```



5.1 A Common Error and how to fix it

```
AttributeError: module 'random' has no attribute 'randint'
```

Python found **your** `random.py` first and imported that instead. The same applies to `math.py`, `csv.py`, `json.py`, `turtle.py`.

Name your files for what they do — `shapes.py`, `physics.py`, `mortgage.py`, `unit_converter.py` — and this never bites you.



Outline

1. What Is a Module?
2. Built-in vs Library Functions
3. Importing modules
4. Your Own File Is a Module
5. The Same-Folder Rule
- 6. Worked Example — Splitting a Program**
7. Recap



6. Worked Example — Splitting a Program

Before — one file, everything mixed together:

```
# guessing_game.py
import random

secret = random.randint(1, 100)
guess = int(input("Guess: "))

if guess < secret:
    print("Too low")
elif guess > secret:
    print("Too high")
else:
    print("Correct - well done!")
```



6. Worked Example — Splitting a Program

After — pure logic in one module, the program in another.

```
# guess_logic.py
"""Pure functions for the guessing game."""
import random

def get_secret_number():
    """Return a random integer between 1 and 100."""
    return random.randint(1, 100)
```



6. Worked Example — Splitting a Program

```
def check_guess(secret, guess):  
    """Return 'Too low', 'Too high' or 'Correct' for the given guess."""  
    if guess < secret:  
        return "Too low"  
    elif guess > secret:  
        return "Too high"  
    else:  
        return "Correct - well done! "
```



6. Worked Example — Splitting a Program

```
# game.py
import guess_logic

secret = guess_logic.get_secret_number()
guess = int(input("Guess: "))

result = guess_logic.check_guess(secret, guess)
print(result)
```



6. Worked Example — Splitting a Program

```
# game.py
import guess_logic

secret = guess_logic.get_secret_number()
guess = int(input("Guess: "))

result = guess_logic.check_guess(secret, guess)
print(result)
```

`check_guess` is a **pure function** in its own file — easy to read, easy to reuse, and easy to test without ever playing the game.



6.1 One Thing to Watch For

When you import a file, Python **runs** it from top to bottom.

```
# shapes.py
import math

def circle_area(radius):
    return math.pi * radius ** 2

print("Loading shapes...")      # this runs on import!
```

```
# test_shapes.py
import shapes                    # prints "Loading shapes..."
```



6.1 One Thing to Watch For

When you import a file, Python **runs** it from top to bottom.

```
# shapes.py
import math

def circle_area(radius):
    return math.pi * radius ** 2

print("Loading shapes...")      # this runs on import!
```

```
# test_shapes.py
import shapes                    # prints "Loading shapes..."
```

Definitions are harmless — but stray `print()` calls and `input()` prompts at module level will fire the moment someone imports your file.



6.1 One Thing to Watch For

For now: keep your module files to `import` lines, constants, and `def` blocks. Put the code that **does** things in the file you actually run.



Outline

1. What Is a Module?
2. Built-in vs Library Functions
3. Importing modules
4. Your Own File Is a Module
5. The Same-Folder Rule
6. Worked Example — Splitting a Program
- 7. Recap**



7. Recap

Task	Code
Use a library module	<code>import math then math.sqrt(16)</code>
Pull out one name	<code>from math import pi then pi</code>
Rename on import	<code>import random as rnd</code>
Use your own file	<code>import shapes then shapes.circle_area(3)</code>



7. Recap

Task	Code
Use a library module	<code>import math then math.sqrt(16)</code>
Pull out one name	<code>from math import pi then pi</code>
Rename on import	<code>import random as rnd</code>
Use your own file	<code>import shapes then shapes.circle_area(3)</code>

- A module is just a `.py` file
- `import` goes at the top of the file
- Your file and the file you import must be in the **same folder**
- Never name a file after a standard library module
- Module files should **define**, not **do**



Thanks for Watching - Any questions?

